Planstudio v2: week one on a Claude Pro subscription

Addendum to `v2-kernel-plan.md` and `v2-agent-specs.md`, written 2026-11-09.

Goal: get the most valuable part of v2 working in one week, using Claude Code on a Pro subscription, with Haiku doing most of the work and a stronger model consulted at fixed checkpoints.

1. What fits in a week, honestly

The whole plan is about 290 agent sessions and 125–190 hours of your review, so it does not fit in a week on any plan. The part that does fit, and that delivers the core value, is:

Week-one MVP: a command-line tool that turns your laser measurements into a solved plan and tells you exactly which numbers conflict and what to measure next.

- **Input:** a `.plan` text file in the DSL from the agent specs (§7), with rooms, clear sizes, relations, laser measurements and openings.
- `plan check` prints diagnostics, for example:
  - E003 "2360 + 90 + 2090 = 4540 vs 4490";
  - W001 "measure Hall.north to Hall.south";
  - W003 "facade not straight, 53 mm".
- `plan solve --json` gives solved room faces, walls, openings and areas.
- `plan svg` gives a simple labelled drawing to look at.
- `plan fmt` rewrites the file in canonical form.
- **Claude Code can drive it directly** through Bash, so you get a measurement assistant in week one. The MCP server comes in week two.

Not in week one:

- the editor, PDF, DXF and `.skp` export;
- MCP and accounts;
- angled walls, and SolidCraft's solver;
- the least-squares spreading of small differences (week one keeps the first-pass solution and reports cycles within tolerance as W004);
- the corner joins on wall outlines (walls are drawn as plain rectangles).

2. What is known about Pro limits (checked 2026-11-09)

- Pro has a **5-hour session limit** and a **weekly limit**. They are shared by claude.ai, Claude Desktop and Claude Code (Anthropic: use Claude Code with your Pro or Max plan).
- The 5-hour window starts at your first message.
- **Anthropic does not publish the sizes.** Check your own with `/status` or `/usage` in Claude Code, or in Settings → Usage.
- **One third-party measurement** put a Pro week at about 145M tokens, 82% of them cached reads, worth roughly $155 at API prices, or about 13 full 5-hour windows (wokey.ai, measured 2026-10-26 to 11-02). It's unverified and accounts may differ.

- **Model choice affects how fast you use the limit**, per Anthropic (usage and rate limits). The exact weights aren't published; the plan below assumes cheaper models cost proportionally less of the limit.
- **Opus on Pro in Claude Code: not confirmed.** An older Anthropic article said Pro couldn't use Opus in Claude Code, and that may have changed. Run `/model` to see what your account offers. If Opus isn't there, use **Sonnet as the reviewer** everywhere this doc says "Opus".
- **Avoid Fable.** One source says Fable needs usage credits on Pro.

Budget (estimate, API-equivalent):

- **Haiku:** about 100–120 short sessions at about $0.15 each, roughly $15–20.
- **Reviewer (Opus or Sonnet):** about 12–15 consults at $1.5–3.5 each, roughly $25–45.
- **Total:** roughly **$40–65**, about 25–45% of the measured Pro week. That leaves room for retries.

In practice the tighter limits are the 5-hour windows and your own time, not the weekly total. Plan for **two working windows a day**.

3. Claude Code setup (day 1, 30 minutes)

3.1 Files to add to the new repo

`CLAUDE.md`. Keep it short, because it is loaded into every session and costs tokens every time:

```
# Planstudio v2 (week one)
Rust workspace. Specs: docs/v2-agent-specs.md (read only the sections a card names).
Rules: docs/v2-agent-specs.md §3. Units: i64 mm. Coordinates: x east, y south.
One card per session. Run: cargo test -q -p core 2>&1 | tail -40
Never edit tests, fixtures or public signatures unless the card says so.
Stuck after two failed attempts: stop and ask for the reviewer.
```

`.claude/agents/reviewer.md`, the stronger model, called only at checkpoints:

```
---
name: reviewer
description: Reviews solver and geometry code against docs/v2-agent-specs.md, writes
pseudo-code for hard cards, unblocks a card that failed twice. Use only when the session
says "consult reviewer".
model: opus
tools: Read, Grep, Glob, Bash
---
You review Planstudio v2 code against docs/v2-agent-specs.md and docs/v2-week1-
pro.md.
Read only the files and spec sections named in the request. Answer with:
1. Verdict: OK / fix needed.
2. The exact defects, with file:line and the spec rule they break.
3. If asked for help on a failing card: the smallest change that makes the named
test pass.
Do not rewrite working code. Keep answers under 300 words.
```

If `/model` shows no Opus, change `model: opus` to `model: sonnet`.

`.claude/agents/implementer.md`, for delegating a card from a longer session:

```
        ---
        name: implementer
        description: Implements one task card from docs/v2-week1-pro.md exactly, then runs
the tests.
        model: haiku
        tools: Read, Edit, Write, Bash, Grep, Glob
        ---
        Implement exactly one card. Read the card, the spec sections it names and the files
it lists.
        Do not change tests or public signatures. Run cargo test -q -p core 2>&1 | tail -40
until green.
        Report: files changed, test result, anything unclear. Stop after two failed
attempts.
```

`.claude/settings.json` sets the default model and pre-allows routine commands, to cut down on permission prompts:

```
        {
          "model": "haiku",
          "permissions": {
            "allow": [
              "Bash(cargo test:*)", "Bash(cargo build:*)", "Bash(cargo fmt:*)",
              "Bash(cargo clippy:*)", "Bash(cargo run:*)", "Bash(git status)",
              "Bash(git diff:*)", "Bash(git add:*)", "Bash(git commit:*)"
            ]
          }
        }
```

Run `/model` once to confirm the names your account accepts, and adjust if they differ.

3.2 Habits that save the most limit

1. **`/clear` after every card.** A fresh session with a small context is much cheaper than a long one, and Haiku does better with less context.
2. **Context pack, not the repo.** Point the session at the card and its spec sections, never "read the codebase".
3. **Trim command output.** Use `cargo test -q ... | tail -40` and `cargo clippy -q`. Full compiler output repeated in a loop is the biggest waste.
4. **No images** in week one. Look at the SVG yourself in a browser.
5. **Reviewer by checkpoint, not by habit.** The reviewer reads only the diff and the spec section, never the whole repo.
6. **Commit after every green card** (`P1-06a: ...`), so a bad attempt is `git checkout .` instead of a long repair conversation.
7. **Watch `/status`.** If a window is nearly used, finish the card and stop; don't start a new one.

4. Week-one cards (Haiku-sized)

The cards from the agent specs (§11) are split smaller here, with pseudo-code for the hard ones. Tier **H** means Haiku implements. Tier **R** means the reviewer writes pseudo-code or reviews. Spec references are to `v2-agent-specs.md`.

ID	Card	Tier	Spec	Done when
W1-01	Workspace with `core` and `cli` crates, CI script (`cargo fmt --check && cargo clippy -q -- -D warnings && cargo test -q`), `CLAUDE.md`, agents and settings from §3	H	§3, §4	`cargo test` runs (zero tests)
W1-02	Fixtures A, B, C and `*.expected.json` exactly as written	H	§7.5–7.6	Files match the spec byte for byte
W1-03	Types: `Mm`, `Source` (+ tolerances), `Quantity`, `Plan`, `Room`, `Side`, `FaceRef`, `Relation`, `Gap`, `Measure`, `Opening`, `WallOverride`, with serde	H	§5.1–5.2	Serde round-trip test per type
W1-04	`Diagnostic`, `Code`, `Severity`, and `message(code, args)` with the templates for E001–E007, E010, W001, W003, W004, I010	H	§8	One test per template, exact strings
W1-05	Lexer: tokens, `1.86m`, comments, line/col spans, E006	H	§7.1, §7.3	12 token tests incl. `Förråd`
W1-06	Parser: statements → `Plan`, indentation rule, defaults	H	§7.1	Fixtures A–C parse; 8 E006 cases with exact text
W1-07	Name resolution: E001 with suggestion (≥ 0.85 Jaro–Winkler), E002	H	§7.4	`Sovrun` → exact §7.4 text
W1-08	Printer, canonical form	H	§7.2	`print(parse(fixture)) == canonical fixture` for A–C (proptest deferred)
W1-R1	**Reviewer checkpoint:** read W1-03…08 diffs against the spec; list defects	R	—	Defects fixed by Haiku in W1-08b
W1-09	Edges from a plan (axis, from-face, to-face, value, tolerance, origin), using the table	H	§6.1	Fixture A gives the edge list in the test exactly
W1-10	Union-find components per axis; anchor; W001 for loose components (text only, no suggestion yet)	H	§6.2 step 2	Synthetic plan with a loose room → W001
W1-11	Spanning tree (BFS in declaration order), assign coordinates	H	§6.2 step 3, pseudo-code §5.1	Fixture A faces equal the §7.6 table
W1-12	Cycle check: residual, cycle path, tolerance sum, E003 data	H	§6.2 step 4, pseudo-code §5.2	Fixtures B and C give E003 with the right numbers
W1-13	E003 text formatting with two chains and suggestions	H	§6.3	Exact §6.3 text for fixture B
W1-14	Within-tolerance cycles → W004 "kept first-pass values" (no spreading in week one)	H	—	Synthetic cycle within tolerance → W004, no E003
W1-R2	**Reviewer checkpoint:** solver review (W1-09…14)	R	§6	OK or listed fixes done
W1-15	Facing-faces finder	H	§9.1, pseudo-code §5.3	Fixture A: list of facing pairs matches test
W1-16	Wall segments per face: split, classify (interior, open, derived, outer)	H	§9.2 steps 1–2	Fixture A segment list matches test
W1-17	Bridge, extend, merge, deduplicate; wall ids	H	§9.2 steps 3–7	Fixture A walls equal the §7.6 list

ID	Card	Tier	Spec	Done when
W1-18	W003 facade step and I010 derived walls	H	§9.2	Fixture A diagnostics exactly W003 + 2× I010
W1-R3	**Reviewer checkpoint:** wall generation review	R	§9	OK or fixes done
W1-19	Openings: placement origin rule, E004, E007, hinge check	H	§9.3	Fixture A: 4 openings in the right walls; 3 error cases
W1-20	Areas and spaces (`open` joins rooms)	H	§9.5	Fixture A areas as §7.6
W1-21	`apply(plan, patch, dry_run)` for DSL patches: add, replace, `-` remove; all-or-nothing	H	§10, §10.1	Patch adding the fixture B line → ok=false, E003, plan unchanged
W1-22	CLI `plan check`, `plan solve --json`, `plan fmt`	H	—	`insta` snapshots for A–C
W1-23	CLI `plan svg`: rooms (light fill, name and area), walls as rectangles (grey), openings (gap plus a line), every element labelled with its id, mm grid every 1 m	H	§5 below	SVG for fixture A opens in a browser and you check it by eye
W1-24	`plan suggest`: W001 with the nearest facing faces as a concrete "measure X to Y" suggestion	H	§6.4	Loose-room fixture → exact suggestion text
W1-R4	**Reviewer checkpoint:** end-to-end run on A–C plus your own lgh 154 file	R	—	Week-one acceptance (§6)

That's 24 Haiku cards and 4 reviewer checkpoints. Expect about 1.5–2× as many Haiku sessions as cards, because of retries.

5. Pseudo-code for the hard cards (written in advance, so Haiku doesn't design)

5.1 Spanning tree (W1-11)

```
for axis in [X, Y]:
  nodes = all face variables on axis (room.west/east for X, room.north/south for Y)
  adj   = map node -> list of (edge_index, neighbour, signed_value)
          # for edge (a→b, v): adj[a] += (i, b, +v); adj[b] += (i, a, -v)
          # keep insertion order = declaration order
  for comp in components(axis) in order of first declared room:
    root = anchor if anchor in comp else first face of comp (by room order, then west/north
first)
    coord[root] = 0 ; parent[root] = None ; depth[root] = 0
    queue = [root] ; tree_edges = {}
    while queue:
      a = queue.pop_front()
      for (i, b, v) in adj[a]:
        if b not visited:
          coord[b] = coord[a] + v ; parent[b] = (a, i) ; depth[b] = depth[a] + 1
          tree_edges.add(i) ; queue.push_back(b)
```

5.2 Cycle check and chains (W1-12, W1-13)

```
for each edge i = (a→b, v, tol) not in tree_edges, in declaration order:
  residual = v - (coord[b] - coord[a])
  path = tree_path(a, b)     # walk up parents from the deeper node until both meet (lowest
common ancestor)
  cycle_tol = tol + sum(edge.tol for edge in path)
  if |residual| > cycle_tol:
      emit E003 { axis, residual, tolerance: cycle_tol,
                  chain_a: path edges ordered from a to b (the implied side),
                  chain_b: [edge i] (the stated side) }
  else if residual != 0:
      emit W004 { edges: path + [i], residual }
```

E003 text from chains:

- **Line 1:** the axis name and `|residual|` mm.
- **Line 2:** chain A as `label value (source)` joined by `+`, then `= sum`.
- **Line 3:** chain B.
- **Line 4:** the allowed difference.
- **Line 5:** suggestions. When chain A contains exactly one wall gap and `wall − |residual|` is at least 40 mm, add "change the wall (g → g−r would fit)".

5.3 Facing faces and segments (W1-15, W1-16)

```
facing(f1, f2) =
  opposite sides (E/W or N/S)
  and f2 lies outward of f1 (f2.coord >= f1.coord for f1=east/south, <= for west/north)
  and 0 <= distance <= 600
  and spans overlap by > 0

for each room face f (sorted by room order, then N, E, S, W):
  cuts = {f.span.start, f.span.end}
  for g facing f: cuts += {g.span.start, g.span.end} clipped to f.span
  for consecutive (s, e) in sorted(cuts):
    covering = facing faces g whose span covers [s, e]
    if covering is empty:            kind = Outer
    elif relation(f, g) is Open:     kind = None
    elif relation(f, g) has Wall(q): kind = Interior(thickness q, source q.source)
    else:                            kind = Interior(thickness = distance, source Assumed,
derived = true)
    segments.push(f, s, e, kind, g)
```

Steps 3–7 of §9.2 (bridge, extend, corner continuation, merge, deduplicate) run on this list in that order.

6. Day-by-day plan

Two 5-hour windows a day; your time about 2–3 hours a day for reviewing, testing and steering.

Day	Window 1	Window 2	Checkpoint
1	Create the repo; §3 setup; W1-01, W1-02, W1-03	W1-04, W1-05	—
2	W1-06, W1-07	W1-08, **W1-R1**, fixes	Parser and printer reviewed
3	W1-09, W1-10, W1-11	W1-12, W1-13	—
4	W1-14, **W1-R2**, fixes	W1-15, W1-16	Solver reviewed
5	W1-17, W1-18	**W1-R3**, fixes, W1-19	Walls reviewed
6	W1-20, W1-21	W1-22, W1-23	—
7	W1-24; write your real lgh 154 and 153 `.plan` files	**W1-R4** end to end; buffer	Acceptance

Stop rules:

- If a card fails twice, `git checkout .`, `/clear`, and ask the reviewer with only the card, the failing test and its output.
- If a day slips, drop W1-24 first, then W1-23 (SVG). Never drop the reviewer checkpoints.

7. Week-one acceptance

- `plan check fixtures/lgh154-a.plan` prints exactly W003 and two I010, no errors.
- `plan check fixtures/lgh154-b.plan` prints the E003 text from the agent specs §6.3, word for word.
- `plan check fixtures/lgh154-c.plan` prints E003 with 53 mm against a 52 mm tolerance.
- `plan solve --json` for fixture A matches `lgh154-a.expected.json` (faces, walls, openings, areas).
- `plan svg` for fixture A shows five labelled rooms, the 121 mm partition, no wall between Hall and Vardagsrum, and the 53 mm facade step.
- You can write a `.plan` file for your next laser session, and Claude Code (on Haiku) can run `plan check` and explain the result to you.

8. After week one

At the same pace on Pro, with the reviewer kept for the hard parts, the rest of the plan takes roughly:

Week	Work
2	MCP server (`apply`, `get`, `check`, `render`) over the CLI's core; PNG renders sized for Claude with labels; weighted least squares (P1-08)
3	PDF export, permit checks port (P3-01)
4	Diagnostic views, photo import of laser readings
5–9	Desktop editor (the biggest phase)
10	Viewer, v1 import, cut-over

That's about **10 weeks in total on Pro** if Haiku keeps up. The editor weeks are where Pro's limits are most likely to bind, because UI work needs more back-and-forth. If you hit the weekly limit repeatedly then, Anthropic's options are usage credits or the Max plan.